Operating System Lab

C++ Implementation of Algorithms

Submitted to: _____

Submitted by: _____

Student ID: _____

Department: Computer Science and Engineering

October 11, 2026

Contents

1 Peterson's Solution **3**

2 Strict Alternation **4**

3 Sleep and Wakeup: Producer-Consumer Problem **5**

4 TSL Instruction **6**

5 XCHG Instruction **7**

6 Semaphore: Producer-Consumer Problem **8**

7 Resource Allocation Graph: Cycle Detection **9**

8 Circular Wait Prevention **10**

9 Banker's Algorithm **11**

10 Single-Instance Deadlock Detection: Wait-for Graph **12**

11 Several-Instance Deadlock Detection **13**

Note: Each program is standalone. Compile each program separately with a C++ compiler that supports C++17, for example:

```
g++ -std=c++17 -pthread program.cpp -o program
```

For the TSL and XCHG examples, C++ atomic operations are used to model the behavior of atomic hardware instructions. The circular-wait prevention program demonstrates the global resource-ordering rule.

1. Peterson's Solution

```cpp
#include <bits/stdc++.h>
using namespace std;

atomic<bool> flag_[2] = {false, false};
atomic<int> turn{0};
int counter = 0;

void process(int id) {
    int other = 1 - id;
    for (int i = 0; i < 5; i++) {
        flag_[id].store(true);
        turn.store(other);
        while (flag_[other].load() && turn.load() == other) {
            this_thread::yield();
        }
        counter++;
        cout << "Process " << id << " entered critical section. Counter = " << counter << '\n';
        flag_[id].store(false);
    }
}

int main() {
    thread t1(process, 0), t2(process, 1);
    t1.join();
    t2.join();
    cout << "Final counter = " << counter << '\n';
    return 0;
}
```

2. Strict Alternation

```cpp
#include <bits/stdc++.h>
using namespace std;

atomic<int> turn{0};
int counter = 0;

void process(int id) {
    for (int i = 0; i < 5; i++) {
        while (turn.load() != id) {
            this_thread::yield();
        }
        counter++;
        cout << "Process " << id << " entered critical section. Counter = " << counter << '\n';
        turn.store(1 - id);
    }
}

int main() {
    thread t1(process, 0), t2(process, 1);
    t1.join();
    t2.join();
    cout << "Final counter = " << counter << '\n';
    return 0;
}
```

3. Sleep and Wakeup: Producer-Consumer Problem

```cpp
#include <bits/stdc++.h>
using namespace std;

queue<int> buffer_;
const int CAPACITY = 5;
mutex m;
condition_variable not_full, not_empty;

void producer() {
    for (int item = 1; item <= 10; item++) {
        unique_lock<mutex> lock(m);
        not_full.wait(lock, [] { return buffer_.size() < CAPACITY; });
        buffer_.push(item);
        cout << "Produced: " << item << '\n';
        lock.unlock();
        not_empty.notify_one();
    }
}

void consumer() {
    for (int i = 1; i <= 10; i++) {
        unique_lock<mutex> lock(m);
        not_empty.wait(lock, [] { return !buffer_.empty(); });
        int item = buffer_.front();
        buffer_.pop();
        cout << "Consumed: " << item << '\n';
        lock.unlock();
        not_full.notify_one();
    }
}

int main() {
    thread p(producer), c(consumer);
    p.join();
    c.join();
    return 0;
}
```

4. TSL Instruction

```cpp
#include <bits/stdc++.h>
using namespace std;

atomic<bool> lock_(false);
int counter = 0;

void process(int id) {
    for (int i = 0; i < 5; i++) {
        bool expected = false;
        while (!lock_.compare_exchange_weak(expected, true)) {
            expected = false;
            this_thread::yield();
        }
        counter++;
        cout << "Process " << id << " entered critical section. Counter = " << counter << '\n';
        lock_.store(false);
    }
}

int main() {
    thread t1(process, 1), t2(process, 2);
    t1.join();
    t2.join();
    cout << "Final counter = " << counter << '\n';
    return 0;
}
```

5. XCHG Instruction

```cpp
#include <bits/stdc++.h>
using namespace std;

atomic<bool> lock_(false);
int counter = 0;

void process(int id) {
    for (int i = 0; i < 5; i++) {
        while (lock_.exchange(true)) {
            this_thread::yield();
        }
        counter++;
        cout << "Process " << id << " entered critical section. Counter = " << counter << '\n';
        lock_.exchange(false);
    }
}

int main() {
    thread t1(process, 1), t2(process, 2);
    t1.join();
    t2.join();
    cout << "Final counter = " << counter << '\n';
    return 0;
}
```

6. Semaphore: Producer-Consumer Problem

```cpp
#include <bits/stdc++.h>
using namespace std;

class Semaphore {
    int value;
    mutex m;
    condition_variable cv;
public:
    Semaphore(int initial) : value(initial) {}
    void wait() {
        unique_lock<mutex> lock(m);
        cv.wait(lock, [&] { return value > 0; });
        value--;
    }
    void signal() {
        {
            lock_guard<mutex> lock(m);
            value++;
        }
        cv.notify_one();
    }
};

queue<int> buffer_;
const int CAPACITY = 5;
Semaphore empty_slots(CAPACITY), full_slots(0), binary_mutex(1);

void producer() {
    for (int item = 1; item <= 10; item++) {
        empty_slots.wait();
        binary_mutex.wait();
        buffer_.push(item);
        cout << "Produced: " << item << '\n';
        binary_mutex.signal();
        full_slots.signal();
    }
}

void consumer() {
    for (int i = 1; i <= 10; i++) {
        full_slots.wait();
        binary_mutex.wait();
        int item = buffer_.front();
        buffer_.pop();
        cout << "Consumed: " << item << '\n';
        binary_mutex.signal();
        empty_slots.signal();
    }
}

int main() {
    thread p(producer), c(consumer);
    p.join();
    c.join();
    return 0;
}
```

7. Resource Allocation Graph: Cycle Detection

```cpp
#include <bits/stdc++.h>
using namespace std;

bool hasCycle(int u, vector<vector<int>>& graph, vector<int>& color) {
    color[u] = 1;
    for (int v : graph[u]) {
        if (color[v] == 1) return true;
        if (color[v] == 0 && hasCycle(v, graph, color)) return true;
    }
    color[u] = 2;
    return false;
}

int main() {
    int processes, resources, edges;
    cin >> processes >> resources >> edges;
    int nodes = processes + resources;
    vector<vector<int>> graph(nodes);
    for (int i = 0; i < edges; i++) {
        char type;
        int p, r;
        cin >> type >> p >> r;
        int resourceNode = processes + r;
        if (type == 'A') graph[resourceNode].push_back(p);
        else if (type == 'R') graph[p].push_back(resourceNode);
    }
    vector<int> color(nodes, 0);
    bool cycle = false;
    for (int i = 0; i < nodes; i++) {
        if (color[i] == 0 && hasCycle(i, graph, color)) {
            cycle = true;
            break;
        }
    }
    cout << (cycle ? "Cycle detected: deadlock may exist" : "No cycle detected") << '\n';
    return 0;
}
```

8. Circular Wait Prevention

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<int> resources(n);
    for (int i = 0; i < n; i++) cin >> resources[i];
    sort(resources.begin(), resources.end());
    resources.erase(unique(resources.begin(), resources.end()), resources.end());
    cout << "Request resources only in increasing order:\n";
    for (int r : resources) cout << r << ' ';
    cout << '\n';
    cout << "Following a global resource order prevents circular wait.\n";
    return 0;
}
```

9. Banker's Algorithm

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, m;
    cin >> n >> m;
    vector<vector<int>> allocation(n, vector<int>(m));
    vector<vector<int>> maximum(n, vector<int>(m));
    vector<vector<int>> need(n, vector<int>(m));
    vector<int> available(m);

    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++) cin >> allocation[i][j];

    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++) cin >> maximum[i][j];

    for (int j = 0; j < m; j++) cin >> available[j];

    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            need[i][j] = maximum[i][j] - allocation[i][j];

    vector<bool> finished(n, false);
    vector<int> safeSequence;
    vector<int> work = available;

    while ((int)safeSequence.size() < n) {
        bool found = false;
        for (int i = 0; i < n; i++) {
            if (finished[i]) continue;
            bool possible = true;
            for (int j = 0; j < m; j++)
                if (need[i][j] > work[j]) possible = false;
            if (possible) {
                for (int j = 0; j < m; j++) work[j] += allocation[i][j];
                finished[i] = true;
                safeSequence.push_back(i);
                found = true;
            }
        }
        if (!found) break;
    }

    if ((int)safeSequence.size() == n) {
        cout << "System is in a safe state.\nSafe sequence: ";
        for (int i = 0; i < n; i++) {
            cout << 'P' << safeSequence[i];
            if (i + 1 < n) cout << " -> ";
        }
        cout << '\n';
    } else {
        cout << "System is not in a safe state.\n";
    }
    return 0;
}
```

10. Single-Instance Deadlock Detection: Wait-for Graph

```cpp
#include <bits/stdc++.h>
using namespace std;

bool dfs(int u, vector<vector<int>>& graph, vector<int>& color) {
    color[u] = 1;
    for (int v : graph[u]) {
        if (color[v] == 1) return true;
        if (color[v] == 0 && dfs(v, graph, color)) return true;
    }
    color[u] = 2;
    return false;
}

int main() {
    int n, e;
    cin >> n >> e;
    vector<vector<int>> graph(n);
    for (int i = 0; i < e; i++) {
        int u, v;
        cin >> u >> v;
        graph[u].push_back(v);
    }
    vector<int> color(n, 0);
    bool cycle = false;
    for (int i = 0; i < n; i++) {
        if (color[i] == 0 && dfs(i, graph, color)) {
            cycle = true;
            break;
        }
    }
    cout << (cycle ? "Deadlock detected" : "No deadlock detected") << '\n';
    return 0;
}
```

11. Several-Instance Deadlock Detection

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, m;
    cin >> n >> m;
    vector<int> available(m);
    vector<vector<int>> allocation(n, vector<int>(m));
    vector<vector<int>> request(n, vector<int>(m));

    for (int j = 0; j < m; j++) cin >> available[j];

    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++) cin >> allocation[i][j];

    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++) cin >> request[i][j];

    vector<int> work = available;
    vector<bool> finish(n, false);

    for (int i = 0; i < n; i++) {
        bool hasAllocation = false;
        for (int j = 0; j < m; j++)
            if (allocation[i][j] != 0) hasAllocation = true;
        finish[i] = !hasAllocation;
    }

    bool changed = true;
    while (changed) {
        changed = false;
        for (int i = 0; i < n; i++) {
            if (finish[i]) continue;
            bool canFinish = true;
            for (int j = 0; j < m; j++)
                if (request[i][j] > work[j]) canFinish = false;
            if (canFinish) {
                for (int j = 0; j < m; j++) work[j] += allocation[i][j];
                finish[i] = true;
                changed = true;
            }
        }
    }

    vector<int> deadlocked;
    for (int i = 0; i < n; i++)
        if (!finish[i]) deadlocked.push_back(i);

    if (deadlocked.empty()) {
        cout << "No deadlock detected.\n";
    } else {
        cout << "Deadlock detected among processes: ";
        for (int p : deadlocked) cout << 'P' << p << ' ';
        cout << '\n';
    }
    return 0;
}
```